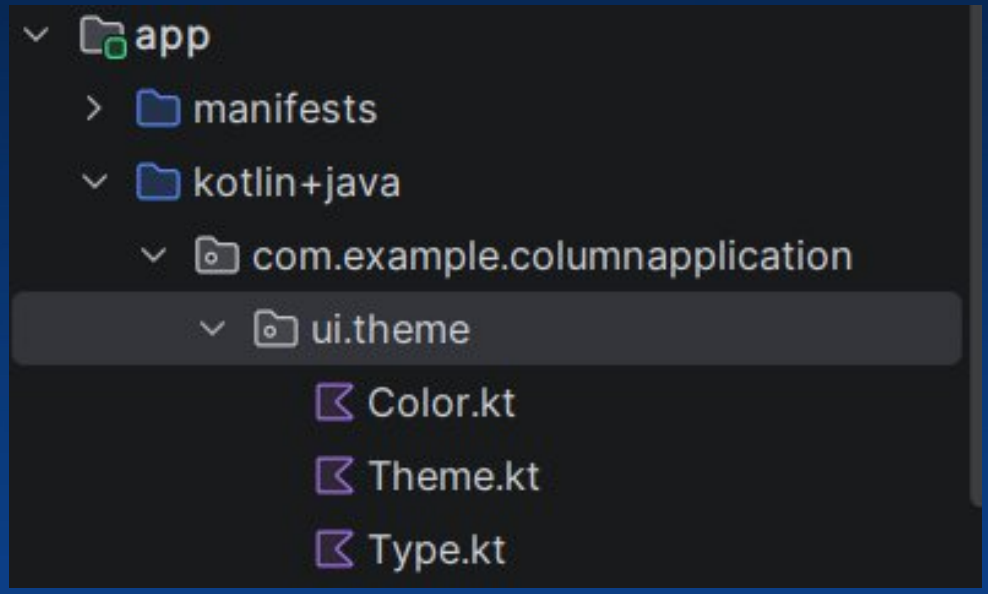
UD2. Scaffold e Intents

0. Índice

1. Material Design
2. Color
3. Tipografía
4. Tema
5. Scaffold
6. Estados
7. Github
8. Intents

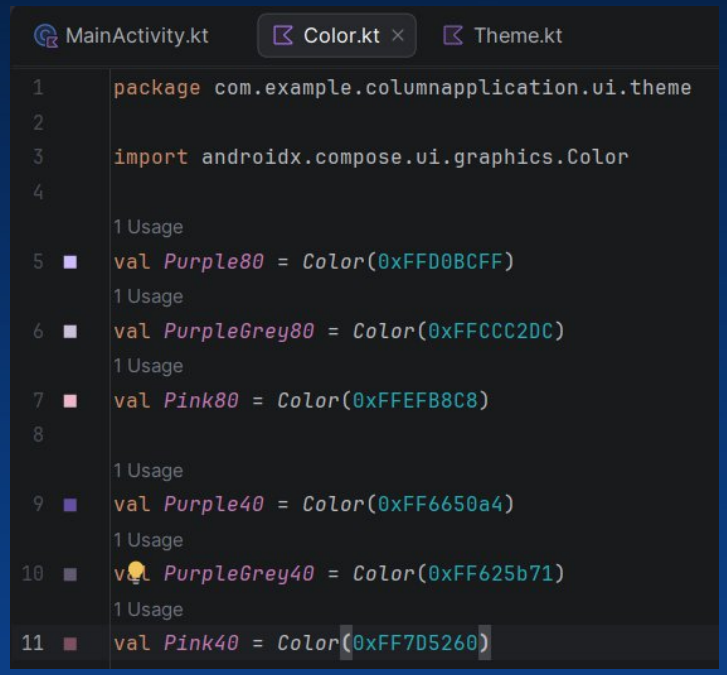
1. Material Design

- Compose está diseñado para admitir los principios de Material Design.
- Material Design ubica dentro de la carpeta ui.theme todos los aspectos relativos al color, tipografía y temas.



2. Color

- Color.kt establece todos los colores que definiremos para nuestra aplicación: fondos, cajas, textos...
- De esta manera será fácilmente modificable si decidimos cambiar los colores de toda la aplicación en una sola sentencia.



3. Tipografía

- Type.kt establece todas las fuentes y estilos que definiremos para nuestra aplicación.
- De esta manera será fácilmente modificable si decidimos cambiar los estilos de toda la aplicación en una sola sentencia.

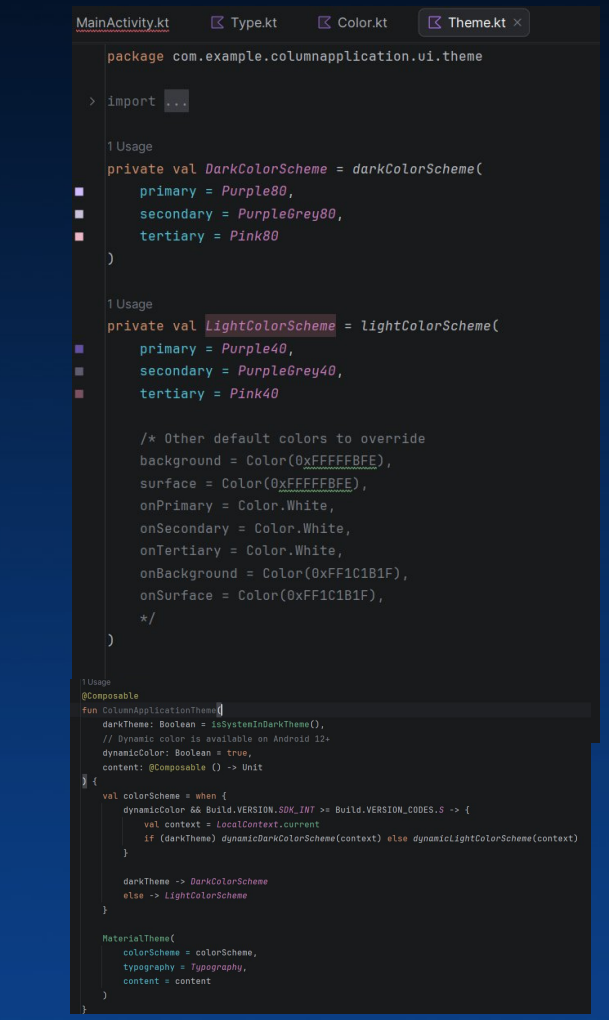
```
package com.example.columnapplication.ui.theme

import ...

// Set of Material typography styles to start with
1 Usage
val Typography = Typography(
    bodyLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    )
    /* Other default text styles to override
    titleLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 22.sp,
        lineHeight = 28.sp,
        letterSpacing = 0.sp
    ),
    labelSmall = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 11.sp,
        lineHeight = 16.sp,
        letterSpacing = 0.5.sp
    )
    */
)
```

4. Tema

- Theme.kt establece los colores y tipografías, entre otras más cosas, que usará nuestra aplicación para los temas LightColorScheme y DarkColorScheme, es decir para cuando usamos nuestra aplicación de día o de noche.



4. Tema

- Ahora vamos a ver como aplicarlo en nuestra Activity:

color = MaterialTheme.colorScheme.primary,
style = MaterialTheme.typography.bodyLarge,

```
Text(
    text = "DI",
    fontSize = 32.sp,
    color = MaterialTheme.colorScheme.primary, style = MaterialTheme.typography.bodyLarge,
    modifier = Modifier.fillMaxWidth()
)
```

5. Scaffold

- En Material Design, un Scaffold (andamiaje) es una estructura fundamental que proporciona una plataforma estandarizada para interfaces de usuario complejas. Une diferentes partes de la IU, como barras de la aplicación y botones de acción flotantes, lo que les da a las apps un aspecto y una sensación coherentes.

- Scaffold es una estructura fundamental en Jetpack Compose que nos proporciona un marco organizativo completo para construir pantallas con elementos de UI estándar.

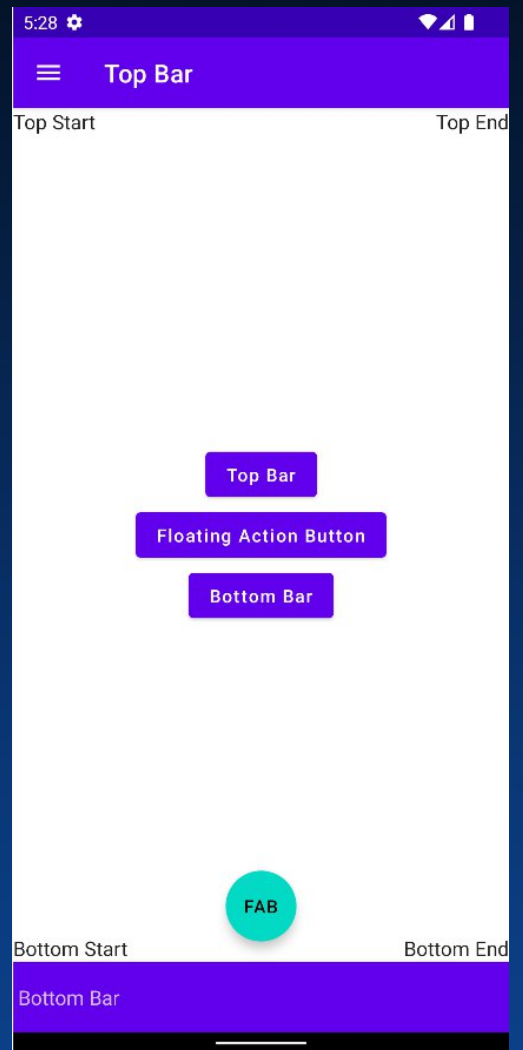
- Actúa como el esqueleto de nuestra aplicación, definiendo las áreas clave donde se colocan los componentes más comunes de Material Design.

- https://developer.android.com/develop/ui/compose/components/scaffold?hl=es-419



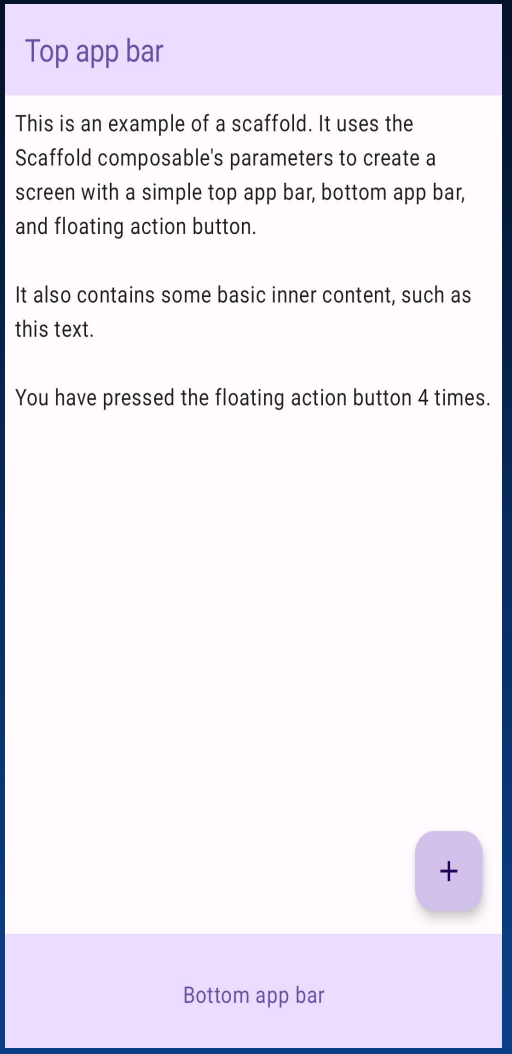
5. Scaffold

- Elementos principales:
  - TopBar: Barra superior de navegación y título de pantalla.
  - BottomBar: Barra inferior de navegación o acciones principales.
  - FloatingActionButton: Botón flotante para la acción más importante.
  - Content: Área principal donde se coloca el contenido de la pantalla.



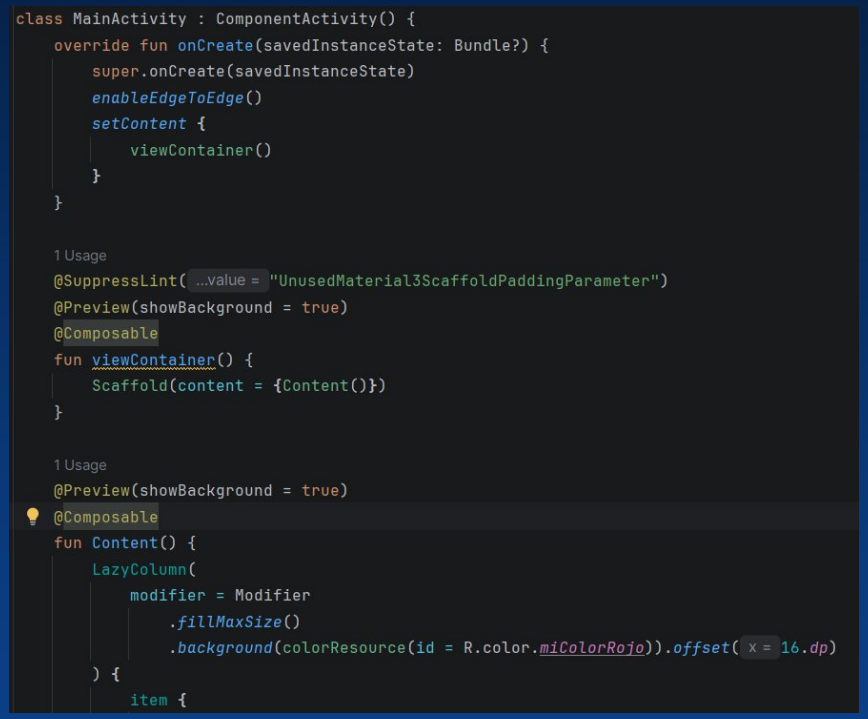
5. Scaffold

- Estructura principal de Scaffold:
  - TopBar: Scaffold(topBar = { ... }, bottomBar = { ... }, floatingActionButton = { ... }) { innerPadding -> // Contenido principal}
- El parámetro innerPadding es crucial: garantiza que el contenido no se superponga con las barras superior e inferior.
- Usos:
  - TopBar: Icono de menú, título, flecha de “volver atrás”, icono de búsqueda, icono de ajustes, icono de compartir.
  - BottomBar: Apps con varias secciones principales, apps con navegacion persistente, inicio, perfil, ajustes, favoritos, notificaciones



5. Scaffold

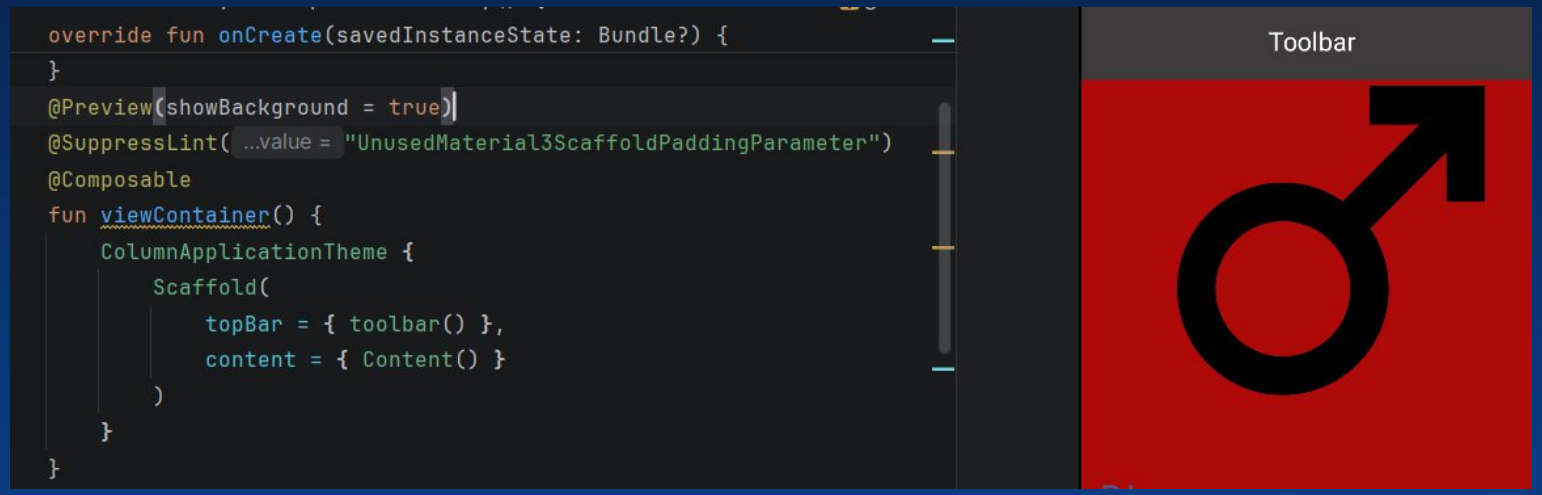
- Vamos a cambiar un poco la estructura de nuestro proyecto para utilizar Scaffold:
  - A la función anterior que contenía nuestras columnas y filas la llamamos Content()
  - Creamos la función viewContainer()
  - Dentro de ella llamamos a Scaffold(content = {Content()})
  - Creamos también la función toolbar()
  - A partir de ahora la única función con @Preview será toolbar()



5. Scaffold

- Vamos definir en la función viewContainer() dos parámetros de Scaffold, la topBar y el content, que llamarán a sus respectivas funciones.

```
@Preview(showBackground = true)
@Composable
fun ViewContainer() {
        ColumnApplicationTheme {
            Scaffold(
                topBar = {toolbar()},
                content = {Content()}
            )
        }
    }
```

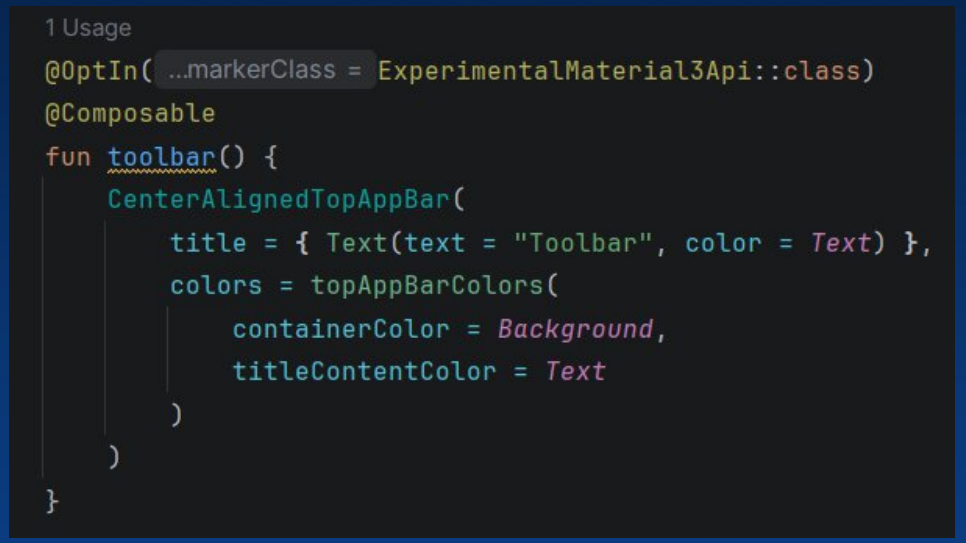


5. Scaffold

- La función toolbar() contendrá todo lo relacionado con el apartado gráfico de nuestra barra de herramientas:

```
@OptIn(ExperimentalMaterial3Api::class)
    @Composable
    fun Toolbar(){

        CenterAlignedTopAppBar(title = { Text(text = "Toolbar", color = Text) },
            colors = topAppBarColors(
                containerColor = Background,
                titleContentColor = Text
            )
        )
    }
}
```

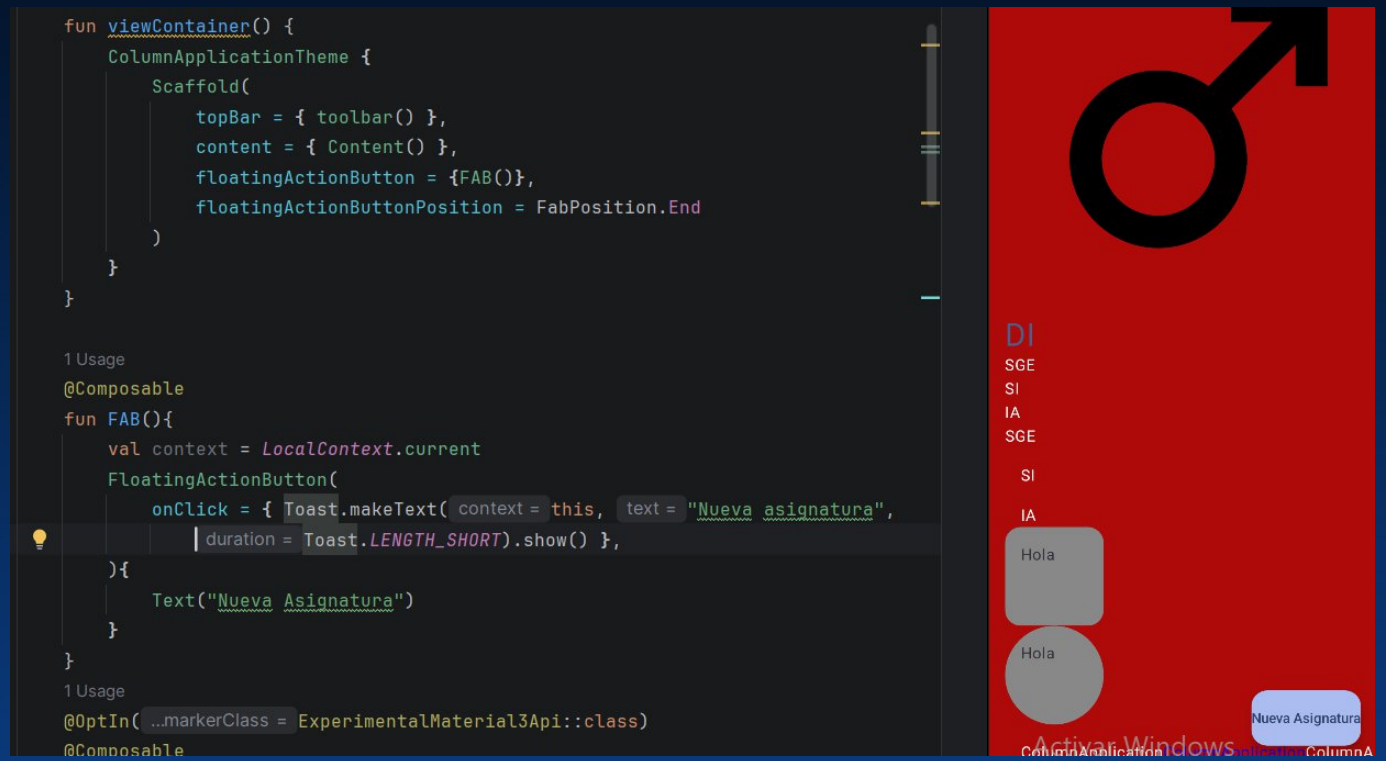


5. Scaffold

- Añadimos un floatingActionButton:

```
fun viewContainer() {
        ColumnApplicationTheme {
          Scaffold(
            topBar = { Toolbar() },
            content = { Content() },
            floatingActionButton = {FAB()},
            floatingActionButtonPosition = FabPosition.End
          )
        }
      }

      @Composable
      fun FAB(){
        val context = LocalContext.current
        FloatingActionButton(
          onClick = { Toast.makeText(this, "Nueva asignatura",
            Toast.LENGTH_SHORT).show() },
        ){
          Text("Nueva Asignatura")
        }
      }
```



6. Estados

- Queremos añadir un contador de likes:

```
@Composable
fun Content() {
    var counter by rememberSaveable { mutableIntStateOf( value = 0) }
```

var counter by rememberSaveable { mutableIntStateOf(0) }

- rememberSaveable sirve para que aunque se cierre la aplicación, el valor perdure.
- mutableIntStateOf(0) crea un estado reactivo que:
  - Mantiene el valor entre recomposiciones gracias a remember.
  - Se inicializa en 0 con mutableIntStateOf(0).
  - Actualiza automáticamente la UI cuando cambia su valor.

6. Estados

- Añadimos una Row que contiene la imagen del corazón y el texto que mostrará el valor de la variable content:

```
item {
    Image(
        modifier = Modifier.fillMaxWidth().height(400.dp).clickable{counter++},
        painter = painterResource(id = R.drawable.male),
        contentDescription = "logo hombre"
    )

    Row (modifier = Modifier.padding(top = 8.dp)){
        Image(
            painter = painterResource(id = R.drawable.heart),
            contentDescription = "icono like",
            colorFilter = ColorFilter.tint(Color.White),
            modifier = Modifier.clickable{counter++}
        )

        Text(text =counter.toString(), color = Color.White, modifier = Modifier.padding(start = 4.dp))
    }
```



- Ejercicios 1-4

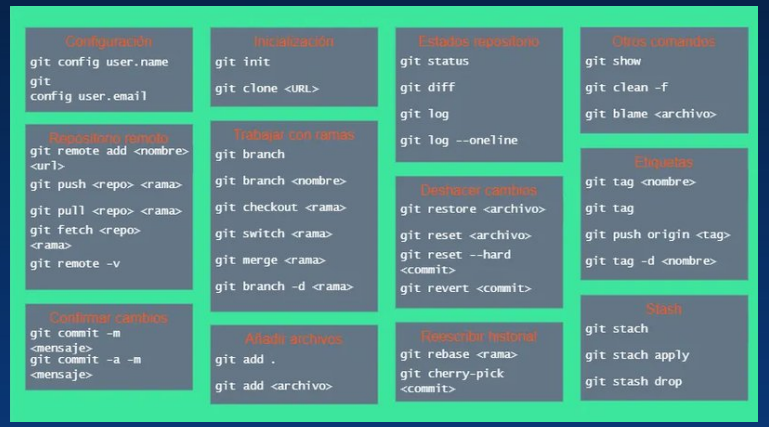
7. Github

- Git: Sistema de control de versiones distribuido que registra todos los cambios en vuestro código.
- GitHub: Plataforma cloud para alojar repositorios Git y facilitar la colaboración en equipo. Proporciona:
  - Control de versiones: Deshacer errores, ver el historial completo de cambios.
  - Colaboración fluida: Trabajar en equipo sin conflictos ni pérdida de código.
  - Respaldo automático: Vuestro código seguro y accesible desde cualquier lugar.



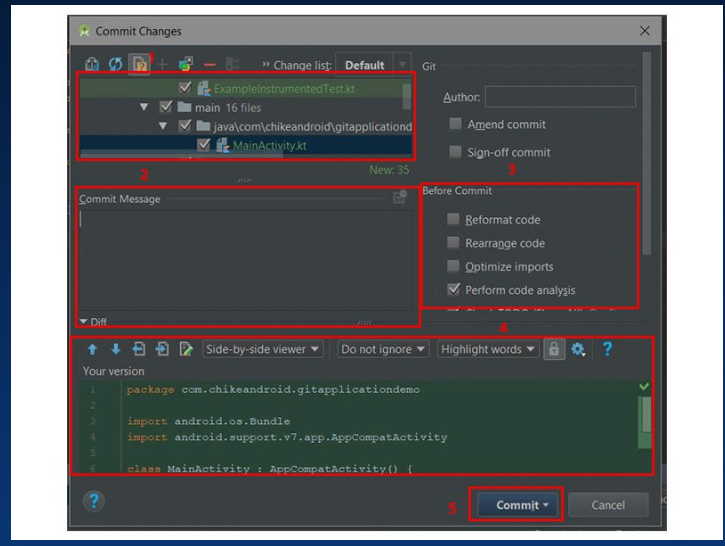
7. Github

- Comandos Git esenciales:
  - git init: Inicializa un repositorio git en vuestro proyecto Android desde cero.
  - git status: Verifica el estado actual: archivos modificados, nuevos o listos para commit.
  - git add: Añade todos los cambios al staging area, preparándolos para el próximo commit.
  - git commit –m: Guarda los cambios con un mensaje descriptivo y claro del trabajo realizado.
  - Git push origin main: sube vuestros cambios al repositorio remoto en GitHub para compartir con el equipo.



7. Github

- Menú VCS: Android Studio incluye Git integrado. No necesitáis la terminal para operaciones básicas.
- Compartir Proyecto: VCS —> Import into Version Control —> Share Projecto on GitHub
- Hacer Commit: Commit (Ctrl + K) Seleccionar archivos, escribir mensaje descriptivo.
- Push a GitHub: Push (Ctrl + Shift + K) Subir cambios al repositorio remoto.



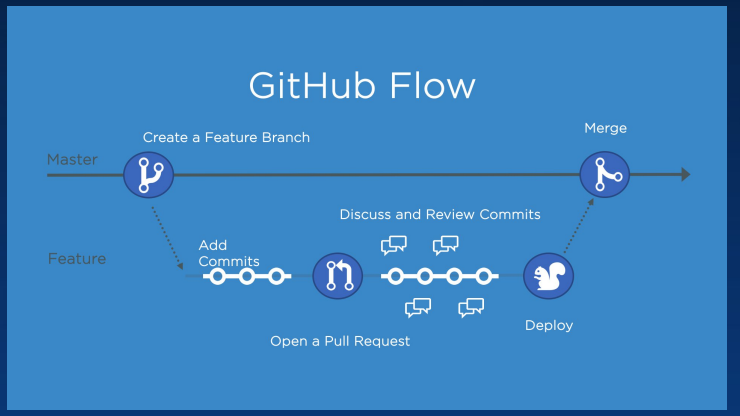
7. Github

- Creando vuestro repositorio en GitHub
  - Crear cuenta GitHub: Registraos en GitHub.com con vuestro email universitario para acceso premium.
  - Nuevo repositorio:
    - Nombre: proyecto-android-apellido-nombre
    - Descripción del proyecto Android
  - Configuración
    - Seleccionar Public o Private según vuestras necesidades.
    - Añadir README.md opcional.
  - Conectar con Android Studio
    - Vincular vuestro IDE con la cuenta de GitHub para sincronización automática



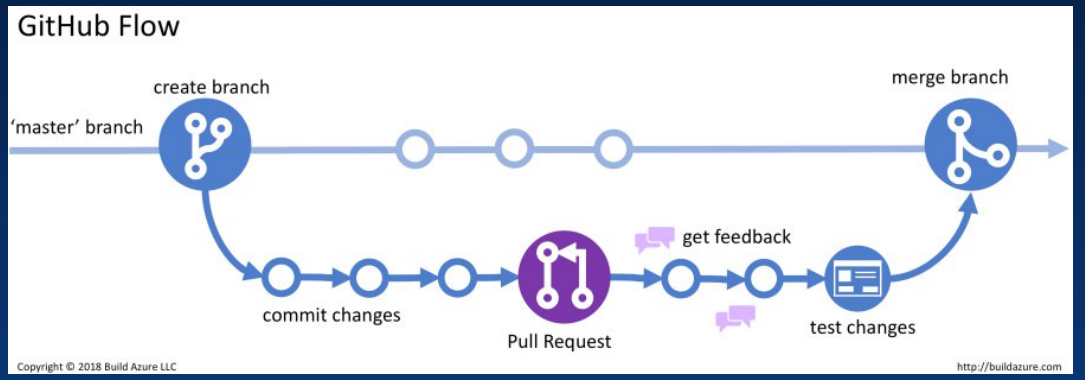
7. Github

- Flujo de trabajo básico:
  - Git pull origin main: Descarga los cambios más recientes del repositorio remoto a vuestra máquina local
  - Desarrollar en Android Studio: Realizad vuestros cambios; nuevas funcionalidades, correcciones de bugs, mejoras...
  - Git add: Añadir todos los archivos modificados al área de preparación.
  - Git commit –m “mensaje”: Guardar cambios con mensaje descriptivo y profesional.
  - Git push origin main: Subir vuestros cambios a GitHub para compartir con el equipo.



7. Github

- Ramas (Branches): Copias independientes de vuestro proyecto para desarrollar nuevas funcionalidades sin afectar el código principal de producción.
  - Crear rama:
    - git branch feature-login
    - En Android Studio: VCS —> Git —> Branches —> New Branch
  - Cambiar de rama:
    - git checkout feature-login
    - En Android Studio: VCS —> Git —> Branches —> Checkout
  - Fusionar cambios:
    - git merge feature-login
    - Cuando la funcionalidad esté lista, fusionar con la rama principal



7. Github

- Ejemplo de README.md

```
# Nombre del Proyecto

Descripción breve del proyecto: qué hace, para qué sirve y cuál es su objetivo principal.

## Tabla de Contenidos

- [Instalación](#instalación)
- [Uso](#uso)
- [Tecnologías](#tecnologías)
- [Contribución](#contribución)
- [Licencia](#licencia)
- [Autores](#autores)

## Instalación

Instrucciones claras y sencillas para instalar y preparar el entorno.

```bash
# Ejemplo de comandos para instalar dependencias o preparar el proyecto
git clone https://github.com/tu_usuario/tu_repositorio.git
cd tu_repositorio
# instalar dependencias
```
```

7. Github

- Anotaciones de Commit

Tipo	Uso
`feat`	Nueva funcionalidad
`fix`	Corrección de errores
`docs`	Cambios en la documentación
`style`	Formato (indentación, espacios) sin cambios de código lógico
`refactor`	Cambios internos sin cambiar funcionalidad
`test`	Añadir o cambiar tests
`chore`	Tareas de mantenimiento (build, dependencias…)

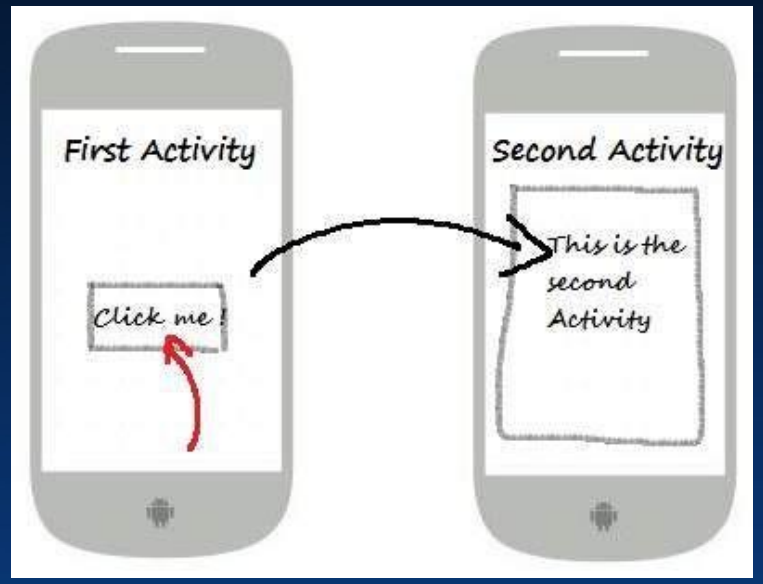
- Ejemplos:

feat: implementar botón para añadir tareas
fix: evitar crash al iniciar sin conexión
docs: actualizar instrucciones de instalación
refactor: dividir lógica de validación en una función aparte

- Ejercicio 5

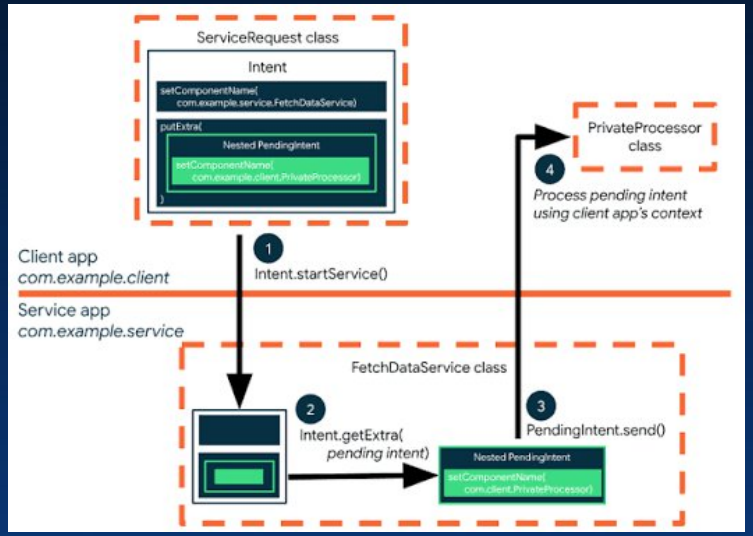
8. Intents

- Intents es el mecanismo fundamental que permite la comunicación entre aplicaciones en el ecosistema Android.
- Un intent es un objeto de mensajería que actúa como un puente de comunicación entre diferentes componentes de aplicaciones android. Permite solicitar acciones y transferir datos de manera eficiente y segura.
- Los intents son esenciales para crear aplicaciones interconectadas que aprovechan las capacidades del sistema Android.



8. Intents

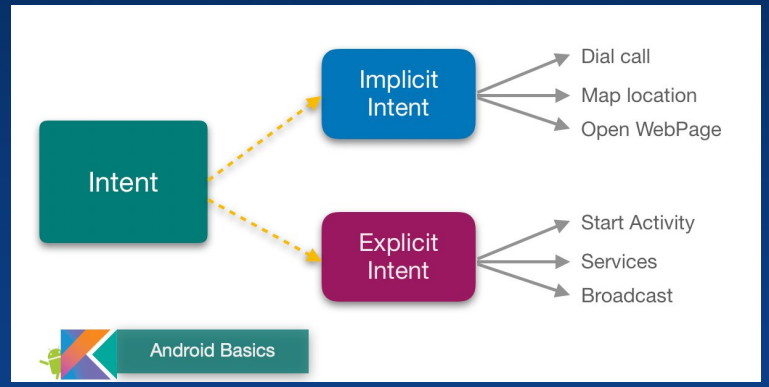
- Funciones principales de los Intents:
  - Iniciar actividades: Abre nuevas pantallas dentro de tu aplicación o navega entre diferentes secciones de la interfaz.
  - Transferir datos: Envía información entre actividades, servicios o incluso aplicaciones externas de forma segura.
  - Solicitar acciones: Delega tareas específicas a otras aplicaciones, como abrir navegadores, enviar correos o compartir contenido.
- Existen dos tipos de intent:
  - Intent explícito
  - Intent implícito



8. Intents

- Intent explícito: Indicas directamente qué componente de tu aplicación debe manejar el intent, especificando la clase exacta.
  - Uso típico: Comunicación interna entre actividades, servicios o receptores dentro de tu propia aplicación.
  - Seguridad: Más seguro porque conoces exactamente quién va a recibir el intent y procesar los datos.

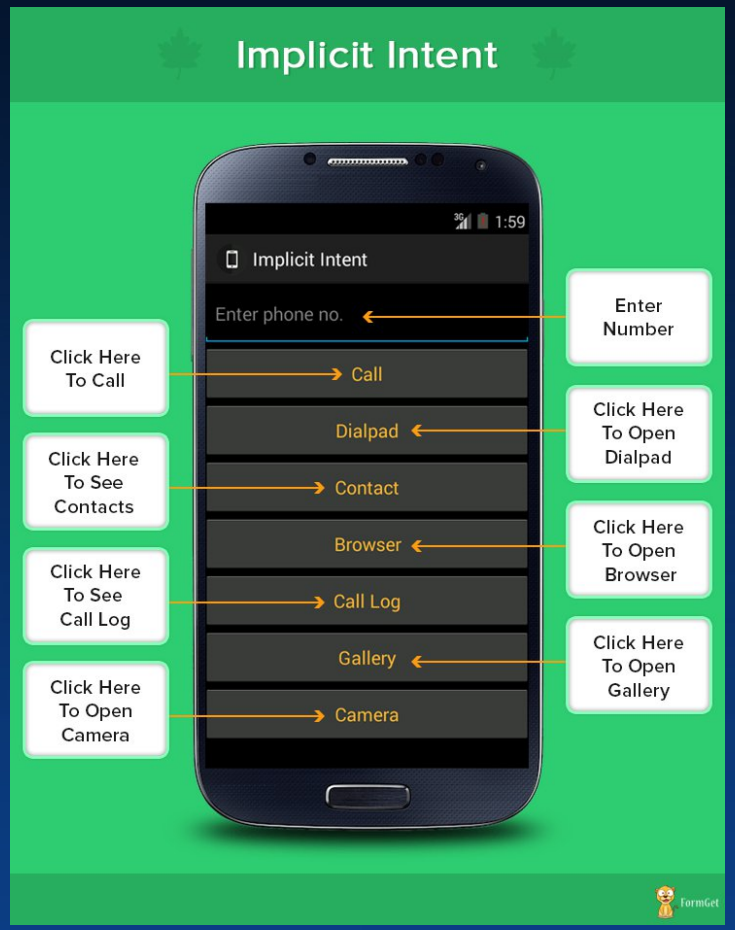
```
Intent intent = new Intent(this, SegundaActividad.class);intent.putExtra("mensaje",
    "Hola desde MainActivity");startActivity(intent);
```



8. Intents

- Intent implícito: Declaras la acción. Sólo especificas qué tipo de acción quieres realizar, sin mencionar una aplicación específica.
- Android examina todas las aplicaciones instaladas que pueden manejar esa acción particular.
- El suario selecciona si hay múltiples opciones, el sistema muestra un diálogo para que el usuario elija su aplicación preferida.

```
Uri webpage = Uri.parse("https://www.ejemplo.com");Intent intent = new
Intent(Intent.ACTION_VIEW, webpage);startActivity(intent);
```

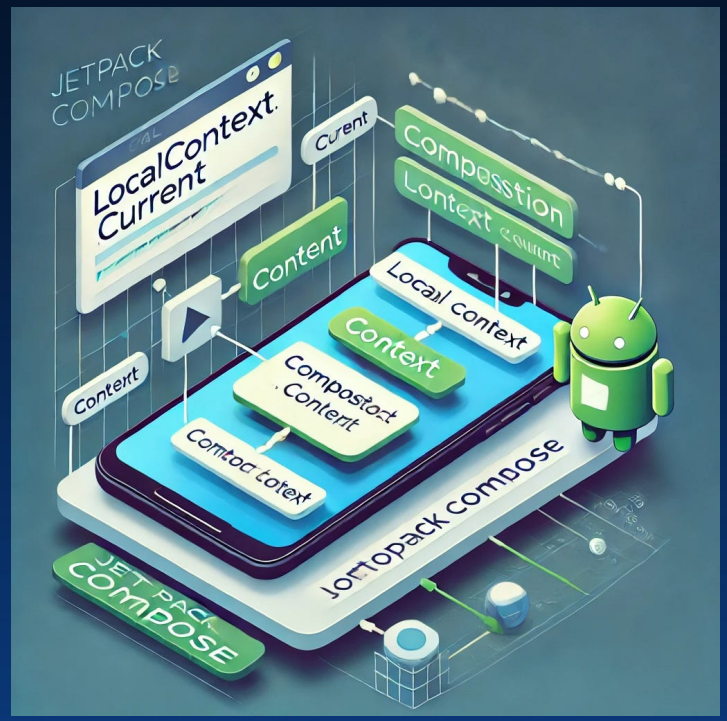


8. Intents

- Configuración inicial en Jetpack Compose
  - Paso esencial: Para utilizar intents en Jetpack Compose, primero debes obtener el contexto actual de la aplicación.

    Val context = LocalContext.current // Accede al contexto actual

  - Esta línea debe aparecer al inicio de tu función Composable y es fundamental para el correcto funcionamiento de los intents.



8. Intents

- Ejemplos prácticos de intents:
  - Envío de correo:

    Intent.ACTION_SENDTO //para abrir aplicaciones de correo electrónico con destinatario y asunto predefinidos.

  - Compartir contenido:

    Intent.ACTION_SEND //para compartir texto, enlaces o archivos con otras aplicaciones instaladas.

  - Abrir navegador:

    Intent.ACTION_VIEW //para cargar URLs en el navegador predeterminado del usuario.

  - Cambiar actividad:

    Intent explícito para navegar entre diferentes pantallas de tu aplicación.



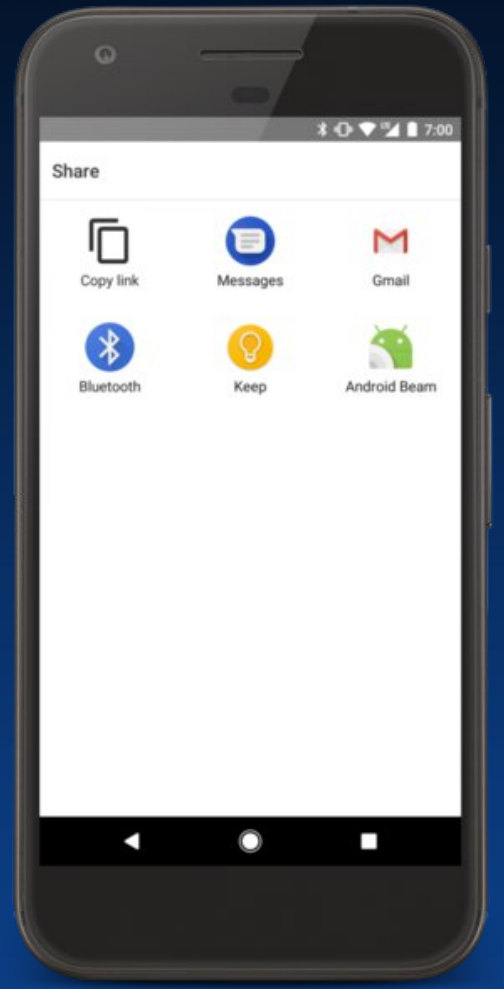
8. Intents

- Acciones de intents más utilizadas:
  - Comunicación:

    ACTION_SENDTO //Enviar correos
    ACTION_SEND //Compartir contenido
    ACTION_CALL //Realizar llamadas
    ACTION_DIAL //Abrir marcador

  - Sistema y archivos:

    ACTION_VIEW //Abrir enlaces
    ACTION_PICK //Seleccionar archivos
    ACTION_EDIT //Editar archivos
    ACTION_MAIN //Actividad principal



8. Intents

- Intent para enviar correo:

```
val context = LocalContext.current

Button(onClick = {
    val intent = Intent( action = Intent.ACTION_SENDTO).apply {
        data = Uri.parse( uriString = "mailto:alumno@ejemplo.com")
        putExtra( name = Intent.EXTRA_SUBJECT, value = "Saludos desde Compose")
        putExtra( name = Intent.EXTRA_TEXT, value = "Hola, este es un mensaje de prueba")
    }
    context.startActivity( intent = intent)
}) {
    Text("Enviar correo")
}
```

8. Intents

- Intent para compartir contenido:

```
Button(onClick = {
    val intent = Intent( action = Intent.ACTION_SEND).apply {
        type = "text/plain"
        putExtra( name = Intent.EXTRA_TEXT,  value = "Mira esta app de Compose genial!")
    }
    context.startActivity( intent = Intent.createChooser( target = intent,  title = "Compartir con"))
}) {
    Text("Compartir contenido")
}
```

8. Intents

- Intent para abrir el navegador:

```
Button(onClick = {
    val url = "https://www.google.es"
    val intent = Intent( action = Intent.ACTION_VIEW, Uri.parse( uriString = url))
    context.startActivity( intent = intent)
}) {
    Text("Abrir navegador")
}
```

8. Intents

- Intent para abrir otra actividad:

```
Button(onClick = {
    val intent = Intent( packageContext = context,  cls = NextActivity::class.java)
    context.startActivity( intent = intent)
}) {
    Text("Abrir otra actividad")

}
```

8. Intents

- Intents personalizados y BroadcastReceivers:

```
Intent intent = new Intent ("com.miapp.ACCION_PERSONALIZADA");
    intent.putExtra("dato", "Hola");sendBroadcast(intent);
```

  - Flexibilidad total: Define acciones personalizadas que se ajusten exactamente a las necesidades de tu aplicación.
  - Comunicación global: Los BroadcastReceivers registrados pueden escuchar estos intents desde cualquier parte de la aplicación.

- Ejercicio 6

